MATH 231 – Linear Algebra I

The Computational Unit, Part 3 ▪ k-Means Clustering

Kiely Hall 326 ▪ Instructor: Kevin Bedoya

About this document. Part 2 placed a new point into groups that were already known. This part has no groups and no labels: the data arrives as a bare set of vectors, and the task is to find the structure in it. One new object is needed — the mean of a set of vectors — and §11 builds it before the algorithm that uses it.

Part	Sections	Subject
1	§1–§4	operation counting, Big-O, floating point, error
2	§5–§10	k-nearest neighbours
3	§11–§15	k-means clustering

A common confusion (Whose §11 is this?). Section numbering runs continuously through the computational unit, so a bare "§7.4" below means §7.4 *of this unit* — Part 2. References into the vector unit or the matrix unit always name them.

Assumed. From the vector unit: $\mathbb{R}^n$, addition and scalar multiplication (§24.2); the Euclidean norm and distance (§2, §14.2); the metric axioms, in particular the triangle inequality **M4** (§14.1); $\langle \mathbf{u}, \mathbf{u} \rangle = \|\mathbf{u}\|^2$ and the expansion $\|\mathbf{u}+\mathbf{v}\|^2 = \|\mathbf{u}\|^2 + 2\langle \mathbf{u}, \mathbf{v} \rangle + \|\mathbf{v}\|^2$ (§24.3). From this unit: Big-O (§2.7), FLOPs (§4.6), feature vectors and datasets (§5.1), the squared-distance shortcut (§7.5), and the distinction between classification and clustering (§9.1).

What you should be able to do afterwards. State the clustering problem; define the mean $\boldsymbol{\mu}$ of a set of vectors, the centroid and the medoid, and say how they differ; prove that the mean minimises the sum of squared distances; write down the k-means objective; run Lloyd's algorithm by hand and write its pseudocode; explain why it terminates and why it may still give a poor answer; count its operations and give its complexity; and name the standard initialisation and speed-ups used in practice.

11. Clustering, and the mean of a set of vectors

11.1. The problem

> Given a set of vectors $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subseteq \mathbb{R}^n$ and an integer k, partition $\mathcal{X}$ into k groups so that points in the same group are close to one another and points in different groups are far apart.

Compare that with §5.2. There is no $\mathbf{q}$, because we are not asking about a new point; there are no labels y_i, because nobody has told us what the groups are. The output is not a prediction but a **partition** of the data itself.

Definition 11.1 (Clustering)**.** A **clustering** of $\mathcal{X}$ into k parts is a partition $\mathcal{C}_1, \dots, \mathcal{C}_k$ of $\mathcal{X}$: every $\mathbf{x}_i$ belongs to exactly one $\mathcal{C}_j$, and no $\mathcal{C}_j$ is empty. Each $\mathcal{C}_j$ is a **cluster**. Equivalently it is an **assignment** function $a : \{1, \dots, N\} \to \{1, \dots, k\}$, with $\mathcal{C}_j = \{\, \mathbf{x}_i : a(i) = j \,\}$.

Two things must be made precise before there is an algorithm: what "close to one another" means numerically — §12.1 — and what a cluster's *centre* is, which is the rest of this section.

11.2. The mean of a set of vectors

The average of a list of numbers is familiar. The same definition works one slot at a time on vectors, and it is nothing more than the arithmetic of the vector unit applied to m vectors at once.

Definition 11.2 (Mean of a set of vectors)**.** Let $S = \{\mathbf{x}_1, \ldots, \mathbf{x}_m\} \subseteq \mathbb{R}^n$ be non-empty. Its **mean** is the vector

$$\boldsymbol{\mu}(S) \;=\; \frac{1}{m}\sum_{i=1}^{m} \mathbf{x}_i \;\in\; \mathbb{R}^n.$$

When S is understood we write simply $\boldsymbol{\mu}$.

Unpacking that: the sum is $m-1$ vector additions, and dividing by m is scalar multiplication by $1/m$, so $\boldsymbol{\mu}$ is a linear combination of the $\mathbf{x}_i$ with every weight equal to $1/m$. Slot by slot,

$$\boldsymbol{\mu} = \Big[\; \frac{1}{m}\sum_{i=1}^{m} x_{i1}, \quad \frac{1}{m}\sum_{i=1}^{m} x_{i2}, \quad \ldots, \quad \frac{1}{m}\sum_{i=1}^{m} x_{in} \;\Big],$$

which says the mean vector is the vector of the componentwise means: average the first coordinate of everything, average the second, and so on. Nothing new is happening in each slot; what is new is doing all n of them at once and regarding the result as a single point.

Example 11.1. For $S = \{[\,1,1\,],\ [\,2,1\,],\ [\,1,2\,]\}$ in $\mathbb{R}^2$,

$$\boldsymbol{\mu} = \tfrac{1}{3}\big([\,1,1\,] + [\,2,1\,] + [\,1,2\,]\big) = \tfrac{1}{3}[\,4,\,4\,] = \big[\,\tfrac{4}{3},\ \tfrac{4}{3}\,\big].$$

Note that $\boldsymbol{\mu} \notin S$: the mean of three points of the set is not one of them. ■

One identity is used twice below and is worth recording now. Summing the deviations from the mean gives zero:

$$\sum_{i=1}^{m}(\mathbf{x}_i - \boldsymbol{\mu}) \;=\; \sum_{i=1}^{m}\mathbf{x}_i \;-\; m\boldsymbol{\mu} \;=\; m\boldsymbol{\mu} - m\boldsymbol{\mu} \;=\; \mathbf{0},$$

which is just the definition of $\boldsymbol{\mu}$ rearranged. It says the mean sits at the balance point of the set: the displacements to the points cancel exactly.

11.3. Centroid

Definition 11.3 (Centroid)**.** The **centroid** of a finite non-empty set $S \subseteq \mathbb{R}^n$ is its mean $\boldsymbol{\mu}(S)$, regarded as a point of $\mathbb{R}^n$ — the geometric centre of the set.

So mean and centroid are the same object under two names, and the names carry different emphasis rather than different content. *Mean* points at the arithmetic: add and divide. *Centroid* points at the geometry: the balance point, the place where the deviations cancel. This document uses $\boldsymbol{\mu}$ for both and says "centroid" when the picture is what matters.

A common confusion (A centroid need not be one of your points). The example above already shows it: the centroid of three grid points was $[\,4/3,\ 4/3\,]$, which is not a grid point. This is usually harmless and occasionally not. If the vectors encode things that cannot be averaged — a category coded as $1, 2, 3$, a patient record, a physical configuration — the centroid is a point in $\mathbb{R}^n$ with no meaning as an object. That is exactly the situation the medoid is for.

11.4. Medoid

Definition 11.4 (Medoid)**.** The **medoid** of a finite non-empty set $S = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ under a metric d is the element of S whose total distance to the others is smallest:

$$\operatorname{medoid}(S) \;=\; \operatorname*{arg\,min}_{\mathbf{x} \in S} \; \sum_{i=1}^{m} d(\mathbf{x}, \mathbf{x}_i).$$

Two differences from the centroid, and both matter.

- The arg min ranges over S, not over $\mathbb{R}^n$. *A medoid is always one of your data points.* It is the most central actual member, rather than a manufactured average.
- It minimises total *distance*, not total *squared* distance. Squaring penalises large deviations disproportionately, so the centroid is dragged towards outliers while the medoid largely ignores them.

Example 11.2. Take $S = \{\mathbf{x}_1 = [\,1, 1\,],\ \mathbf{x}_2 = [\,2, 1\,],\ \mathbf{x}_3 = [\,1, 2\,]\}$ again, with Euclidean distance. The pairwise distances are $d(\mathbf{x}_1, \mathbf{x}_2) = 1$, $d(\mathbf{x}_1, \mathbf{x}_3) = 1$ and $d(\mathbf{x}_2, \mathbf{x}_3) = \sqrt{2} \approx 1.414$, so the total distances are

$$\mathbf{x}_1 :\ 0 + 1 + 1 = 2, \qquad \mathbf{x}_2 :\ 1 + 0 + 1.414 = 2.414, \qquad \mathbf{x}_3 :\ 1 + 1.414 + 0 = 2.414.$$

The medoid is $\mathbf{x}_1 = [\,1, 1\,]$, an actual member of S, whereas the centroid was $[\,4/3,\ 4/3\,]$, which is not. ■

Remark. Computing a medoid costs more than computing a centroid, and by a lot. The mean needs one pass over the m points; the medoid as defined needs all $\binom{m}{2}$ pairwise distances, which is $O(m^2 n)$. That asymmetry is why k-means is the default and k-medoids (§15.2) is the fallback used when the centroid would be meaningless.

11.5. Why the mean is the right centre

Calling $\boldsymbol{\mu}$ the "centre" has so far been a matter of vocabulary. Here is the theorem that earns it, and it is the result the whole algorithm rests on.

Theorem 11.1 (The mean minimises the sum of squared distances)**.** *Let $S = \{\mathbf{x}_1, \dots, \mathbf{x}_m\} \subseteq \mathbb{R}^n$ be non-empty with mean $\boldsymbol{\mu}$. Then for every $\mathbf{c} \in \mathbb{R}^n$,*

$$\sum_{i=1}^{m} \|\mathbf{x}_i - \mathbf{c}\|^2 \;=\; \sum_{i=1}^{m} \|\mathbf{x}_i - \boldsymbol{\mu}\|^2 \;+\; m\,\|\boldsymbol{\mu} - \mathbf{c}\|^2.$$

Consequently the left-hand side is minimised uniquely at $\mathbf{c} = \boldsymbol{\mu}$.

Proof. Split each displacement through the mean: $\mathbf{x}_i - \mathbf{c} = (\mathbf{x}_i - \boldsymbol{\mu}) + (\boldsymbol{\mu} - \mathbf{c})$. By the expansion of $\|\mathbf{u} + \mathbf{v}\|^2$ from the vector unit, §24.3,

$$\|\mathbf{x}_i - \mathbf{c}\|^2 = \|\mathbf{x}_i - \boldsymbol{\mu}\|^2 + 2\big\langle \mathbf{x}_i - \boldsymbol{\mu},\ \boldsymbol{\mu} - \mathbf{c} \big\rangle + \|\boldsymbol{\mu} - \mathbf{c}\|^2.$$

Sum over $i = 1, \dots, m$. The last term does not depend on i and contributes $m\|\boldsymbol{\mu} - \mathbf{c}\|^2$. For the middle term, the dot product is linear in its first argument (the vector unit, §11.8), so the m cross terms collect into

$$2\Big\langle \sum_{i=1}^{m} (\mathbf{x}_i - \boldsymbol{\mu}),\ \boldsymbol{\mu} - \mathbf{c} \Big\rangle = 2\big\langle \mathbf{0},\ \boldsymbol{\mu} - \mathbf{c} \big\rangle = 0,$$

using the balance identity of §11.2. That leaves exactly the claimed equality.

For the consequence: the first term on the right does not involve $\mathbf{c}$ at all, and the second is $m\|\boldsymbol{\mu} - \mathbf{c}\|^2 \geq 0$ with equality precisely when $\boldsymbol{\mu} - \mathbf{c} = \mathbf{0}$, by the norm axiom **N1** of the vector unit, §15.1. So the total is smallest exactly when $\mathbf{c} = \boldsymbol{\mu}$. □

Remark (What the theorem buys). Read the identity as an accounting statement. The cost of using *any* centre $\mathbf{c}$ splits into two independent pieces: an irreducible part, the spread of the set about its own mean, which no choice of centre can improve; and a penalty $m\|\boldsymbol{\mu} - \mathbf{c}\|^2$ paid for placing the centre away from the mean, growing with the square of how far off you are and with how many points must live with the error.

This is why §12.3 line 8 can simply write down the mean. Given a fixed group of points, choosing their optimal centre is not a search — there is a formula, and the theorem says it is exactly right.

11.6. Three centres, compared

Centre	Minimises	Lives in	Cost
Mean / centroid $\boldsymbol{\mu}$	$\sum_i \|\mathbf{x}_i - \mathbf{c}\|^2$	all of $\mathbb{R}^n$	$O(mn)$, one pass
Medoid	$\sum_i d(\mathbf{x}_i, \mathbf{c})$	the set S itself	$O(m^2 n)$
Geometric median	$\sum_i \|\mathbf{x}_i - \mathbf{c}\|$	all of $\mathbb{R}^n$	iterative; no closed form

The middle column is the whole story. All three are "the centre" of S; they differ in what they are the centre *with respect to*. Squared distance gives the mean and a formula; unsquared distance gives the medoid or the geometric median, both more resistant to outliers and both more expensive. The geometric median is listed to complete the contrast and is not used below.

12. The algorithm

12.1. What we are trying to minimise

"Points in the same group should be close to one another" now becomes a number. Give each cluster $\mathcal{C}_j$ a centre $\boldsymbol{\mu}_j$ and add up how far every point is from the centre of its own cluster.

Definition 12.1 (The k-means objective)**.** For a clustering $\mathcal{C}_1, \ldots, \mathcal{C}_k$ of $\mathcal{X}$ with centres $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$, the **within-cluster sum of squares** is

$$J \;=\; \sum_{j=1}^{k} \sum_{\mathbf{x} \in \mathcal{C}_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|^2.$$

It is also called the **inertia**. The k-means problem is to choose the partition and the centres so as to make J as small as possible.

Squared distances are used, not distances, for two reasons that have both now been seen: it removes the square roots (§7.5), and it is the quantity the mean minimises (§11.5). Those two facts are what make the algorithm below both cheap and correct.

A common confusion (Minimising J exactly is out of reach). It is tempting to read the definition as a recipe. It is not. There are $k^N/k!$ or so ways to partition N points into k groups, and finding the partition with the smallest J is *NP-hard* — no algorithm is known that solves it exactly in

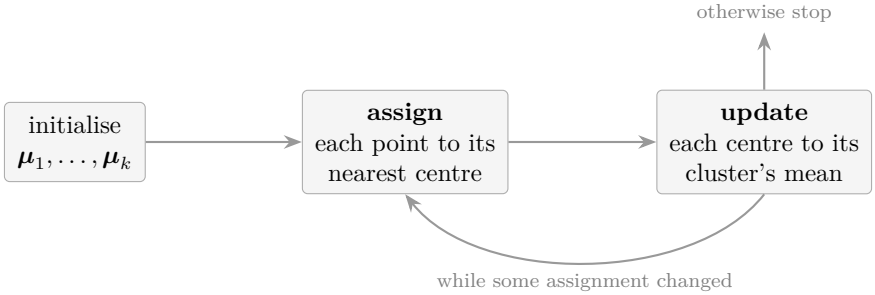
reasonable time, and none is expected. Everything from here is a *heuristic*: a procedure that reliably finds a good clustering and offers no guarantee that it is the best one. §14.1 is about the consequences.

12.2. Walking through it

The algorithm — **Lloyd's algorithm**, though everyone calls it "k-means" — comes from a single observation. The objective J depends on two things, the assignment and the centres, and although optimising both at once is hopeless, optimising *either one with the other held fixed* is easy.

- **Centres fixed, choose the assignment.** Each point contributes $\|\mathbf{x} - \boldsymbol{\mu}_{a(i)}\|^2$ to J and its contribution depends on no other point. So every point should go to the nearest centre. That is k distance computations per point, and no search.
- **Assignment fixed, choose the centres.** Now J splits into k independent sums, one per cluster, and §11.5 says each is minimised by putting the centre at the mean of that cluster. Again a formula, and no search.

So alternate. Assign, then update, then assign again, until nothing changes.



12.3. Pseudocode

---

k-means (Lloyd's algorithm)

Input: data $\mathcal{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \subseteq \mathbb{R}^n$; an integer k with $1 \le k \le N$. *Output:* centres $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$ and an assignment $a : \{1, \ldots, N\} \to \{1, \ldots, k\}$.

---

choose initial centres $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$ — *e.g. k distinct points of $\mathcal{X}$ at random*
repeat
  for $i = 1$ **to** N **do** — *assignment step*
    $a(i) \leftarrow \arg\min_{1 \le j \le k} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$
  end for
  for $j = 1$ **to** k **do** — *update step*
    $\mathcal{C}_j \leftarrow \{\, \mathbf{x}_i \,:\, a(i) = j \,\}$
    $\boldsymbol{\mu}_j \leftarrow \dfrac{1}{|\mathcal{C}_j|} \displaystyle\sum_{\mathbf{x} \in \mathcal{C}_j} \mathbf{x}$ — *the mean, by §11.5*
  end for
until no $a(i)$ changed during this pass
return $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$ and a

---

That is the whole algorithm: two loops, one formula each. Line 4 is k distance comparisons; line 8 is an average. Nothing else happens.

Remark. Line 4 uses $\|\cdot\|^2$ rather than $\|\cdot\|$, which is the shortcut of §7.5 arriving for the second time: the arg min depends only on the ordering of the distances, squaring preserves that ordering, and so Nk square roots per iteration are avoided. Here there is an additional reason to prefer the square, and it is not about speed — J itself is defined with squares, so line 4 is minimising exactly the quantity the algorithm reports.

12.4. Why it stops

Nothing in the pseudocode obviously terminates: line 2 says "repeat". Here is why it must.

- **Neither step can increase J.** The assignment step moves each point to a centre at least as close as its current one, so its contribution to J does not rise. The update step replaces each centre by its cluster's mean, which by §11.5 is the best possible centre for that cluster, so each cluster's contribution does not rise either.
- **There are finitely many assignments.** At most k^N of them — an enormous number, but a finite one. And each assignment determines its centres, hence determines J.

Put those together. If a pass changes some $a(i)$, then J strictly decreases, so that assignment can never recur. A sequence of strictly decreasing values drawn from a finite set must stop, so after finitely many passes no assignment changes and line 10 fires.

A common confusion (Terminating is not the same as being right). The argument shows the algorithm halts and that it halts somewhere no single step can improve. It does *not* show it halts at the best clustering, and in general it does not. What it reaches is a *local* minimum: a configuration where moving one point or nudging one centre makes things worse, while some entirely different partition might have a far smaller J. §14.1 is about living with that.

12.5. A worked example

Example 12.1. Six points in $\mathbb{R}^2$, and $k = 2$:

$$\mathbf{x}_1 = [\,1, 1\,], \quad \mathbf{x}_2 = [\,2, 1\,], \quad \mathbf{x}_3 = [\,1, 2\,], \qquad \mathbf{x}_4 = [\,8, 8\,], \quad \mathbf{x}_5 = [\,9, 8\,], \quad \mathbf{x}_6 = [\,8, 9\,].$$

The two groups are obvious to the eye. To make the run interesting, initialise *badly*: put both centres inside the left-hand group,

$$\boldsymbol{\mu}_1 = [\,1, 1\,], \qquad \boldsymbol{\mu}_2 = [\,2, 1\,].$$

Iteration 1, assignment

Squared distances to each centre, and the winner:

Point	$\|\mathbf{x}-\boldsymbol{\mu}_1\|^2$	$\|\mathbf{x}-\boldsymbol{\mu}_2\|^2$	$a(i)$
$\mathbf{x}_1=[1,1]$	0	1	1
$\mathbf{x}_2=[2,1]$	1	0	2
$\mathbf{x}_3=[1,2]$	1	2	1
$\mathbf{x}_4=[8,8]$	98	85	2
$\mathbf{x}_5=[9,8]$	113	98	2
$\mathbf{x}_6=[8,9]$	113	100	2

So $\mathcal{C}_1=\{\mathbf{x}_1,\mathbf{x}_3\}$ and $\mathcal{C}_2=\{\mathbf{x}_2,\mathbf{x}_4,\mathbf{x}_5,\mathbf{x}_6\}$ — a bad clustering, which is what a bad initialisation buys.

Iteration 1, update

$\boldsymbol{\mu}_1=\frac{1}{2}\big([1,1]+[1,2]\big)=[\,1,\ 1.5\,]$, $\quad\boldsymbol{\mu}_2=\frac{1}{4}\big([2,1]+[8,8]+[9,8]+[8,9]\big)=\big[\,\frac{27}{4},\ \frac{26}{4}\,\big]=[\,6.75,\ 6.5\,]$.
The second centre has been dragged out of the left group towards the right one, because four of its six points live there. At this stage

$$J=\underbrace{0.25+0.25}_{\mathcal{C}_1}+\underbrace{52.8125+3.8125+7.3125+7.8125}_{\mathcal{C}_2}=72.25.$$

Iteration 2, assignment

Recompute against the new centres $[\,1,1.5\,]$ and $[\,6.75,6.5\,]$:

Point	$\|\mathbf{x}-\boldsymbol{\mu}_1\|^2$	$\|\mathbf{x}-\boldsymbol{\mu}_2\|^2$	$a(i)$	
$\mathbf{x}_1=[1,1]$	0.25	63.31	1	
$\mathbf{x}_2=[2,1]$	1.25	52.81	1	(changed)
$\mathbf{x}_3=[1,2]$	0.25	53.31	1	
$\mathbf{x}_4=[8,8]$	91.25	3.81	2	
$\mathbf{x}_5=[9,8]$	106.25	7.31	2	
$\mathbf{x}_6=[8,9]$	105.25	7.81	2	

One assignment changed — $\mathbf{x}_2$ has come home — so the loop continues. Now $\mathcal{C}_1=\{\mathbf{x}_1,\mathbf{x}_2,\mathbf{x}_3\}$ and $\mathcal{C}_2=\{\mathbf{x}_4,\mathbf{x}_5,\mathbf{x}_6\}$.

Iteration 2, update

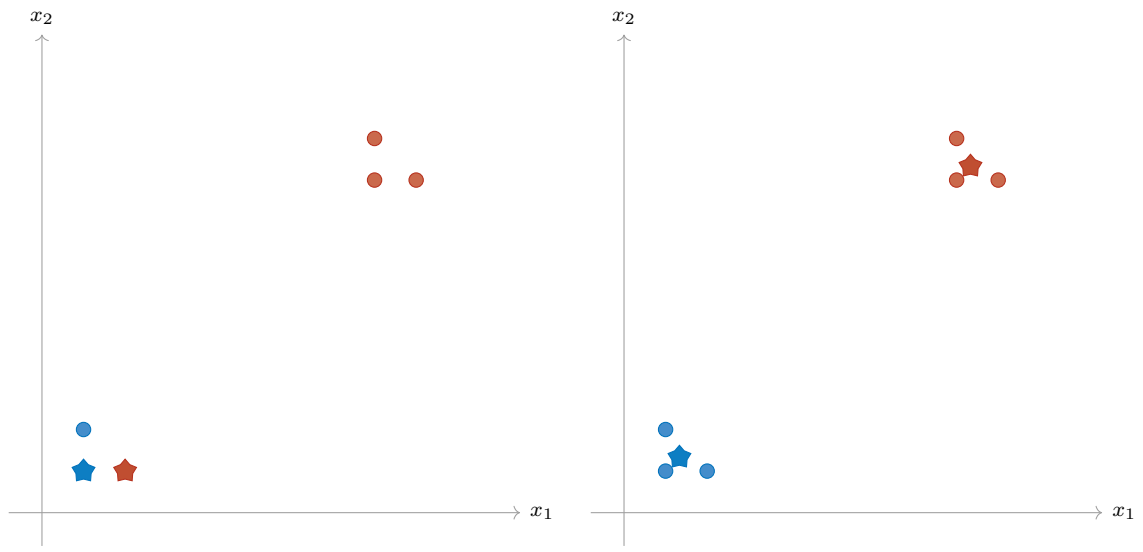
$$\boldsymbol{\mu}_1=\tfrac{1}{3}\big([1,1]+[2,1]+[1,2]\big)=\big[\,\tfrac{4}{3},\ \tfrac{4}{3}\,\big],\qquad\boldsymbol{\mu}_2=\tfrac{1}{3}\big([8,8]+[9,8]+[8,9]\big)=\big[\,\tfrac{25}{3},\ \tfrac{25}{3}\,\big].$$

Iteration 3, assignment

Every point of the left group is now within squared distance $5/9$ of $\boldsymbol{\mu}_1$ and more than 100 from $\boldsymbol{\mu}_2$, and symmetrically on the right. No assignment changes, so line 10 fires and the algorithm returns. The final objective is

$$J=\underbrace{\tfrac{2}{9}+\tfrac{5}{9}+\tfrac{5}{9}}_{\mathcal{C}_1}+\underbrace{\tfrac{2}{9}+\tfrac{5}{9}+\tfrac{5}{9}}_{\mathcal{C}_2}=\tfrac{12}{9}+\tfrac{12}{9}=\tfrac{8}{3}\approx 2.67.$$

Against 72.25 after the first pass, and monotonically decreasing throughout, exactly as §12.4 requires. ■



after iteration 1's assignment

at termination

Stars are centres, discs are data points, and colour is the cluster assignment. On the left both centres sit in the bottom corner and four points have been swept into the red cluster; on the right each centre has settled at the mean of its own group.

13. What it costs

Counting in FLOPs (§4.6), for one iteration on N points in $\mathbb{R}^n$ with k clusters.

13.1. The assignment step

Line 4 computes, for each of the N points, its squared distance to each of the k centres. One squared distance is $3n - 1$ FLOPs by §7.5 — n subtractions, n mults, $n - 1$ adds — so

$$\text{assignment} \;=\; Nk(3n-1)\ \text{FLOPs} \;=\; O(Nkn).$$

Choosing the smallest of k numbers is $k - 1$ comparisons per point, $N(k - 1)$ in total, which is $O(Nk)$ and negligible beside $O(Nkn)$.

13.2. The update step

Line 8 totals each cluster and divides. Every point is added into exactly one running total, and adding a vector of $\mathbb{R}^n$ costs n adds (§1.1), so all the summing together is Nn adds — not Nkn, because each point belongs to one cluster only. Then each of the k centres is divided through by its size, n divisions each. So

$$\text{update} \;=\; Nn + kn\ \text{FLOPs} \;=\; O(Nn),$$

using $k \le N$. The update is cheaper than the assignment by a factor of k.

13.3. The total

Per iteration the assignment step dominates:

$$Nk(3n-1) + Nn + kn \;=\; O(Nkn),$$

and if the algorithm runs for T iterations,

$$\boxed{k\text{-means costs } O(T\,N\,k\,n).}$$

Quantity	**Cost**	**Comment**
Assignment, one iteration	$O(Nkn)$	Nk distances of $O(n)$ each
Update, one iteration	$O(Nn)$	one pass over the data
One iteration	$O(Nkn)$	assignment dominates
Whole run	$O(TNkn)$	T = number of iterations
Memory	$O(Nn + kn)$	the data, plus k centres

Linear in each of N, k, n and T separately, which is why the algorithm scales to large datasets. Compare k-NN, which was $O(Nn)$ per *query* and had no training cost at all: k-means pays its cost once, up front, and afterwards labelling a new point costs only $O(kn)$ — the distance to each of k centres. It is an eager learner in the sense of §7.4, where k-NN is a lazy one.

Remark (The one term that is not honest). T is doing something the other three are not. N, k and n are inputs you know before you start; T is however many iterations the algorithm happens to need, which nobody knows in advance. In practice T is small — typically tens — and implementations cap it anyway with a maximum-iteration parameter. In theory it can be far worse: examples are known where Lloyd's algorithm takes a number of iterations exponential in N. Quoting $O(TNkn)$ and then treating T as a modest constant is standard, and it is a convention rather than a theorem.

14. What goes wrong

14.1. Initialisation, and local minima

The example of §12.5 recovered from a poor start. It need not have. Because J only ever decreases, the algorithm can never climb out of a bad basin it falls into, and the clustering it returns can depend entirely on where the centres began.

Two standard responses, and they are used together.

- **Restart.** Run the whole algorithm several times from different random initialisations and keep the run with the smallest final J. This multiplies the cost by the number of restarts and is worth it.
- **Initialise better.** Rather than choosing k starting points uniformly at random, choose them to be spread out. That is **k-means++**, §15.2, and it is the default everywhere.

14.2. Choosing k

The algorithm takes k as an input, and the data rarely announces it.

A common confusion (You cannot choose k by minimising J). It is the obvious idea and it fails completely. J decreases every time k increases — more centres means every point can be closer to one — and at $k = N$ every point is its own cluster and $J = 0$. Minimising J over k always returns $k = N$, which is not a clustering of anything.

What is done instead:

- **The elbow method.** Plot J against k. The curve falls steeply while k is too small and then flattens once the real groups have been found; the "elbow" where the slope changes is taken as the answer. It is a judgement call read off a graph, and it is often ambiguous.
- **The silhouette score.** For each point, compare its average distance to its own cluster with its average distance to the nearest other cluster, and average the result over all points. Unlike J this does not improve automatically with k, so it can be maximised.
- **Knowing the answer.** Often k is set by the application — a palette of 256 colours, a marketing team that can run four campaigns — and no statistics are needed.

14.3. The shape assumption

Look again at line 4: a point joins the cluster whose centre is nearest. That single line decides the geometry of everything the algorithm can produce. The set of points closer to $\boldsymbol{\mu}_j$ than to any other centre is bounded by flat surfaces — the perpendicular bisectors between centres — so *k-means always cuts the space into straight-sided convex pieces*. It is structurally incapable of returning a cluster shaped like a ring or a crescent, however obvious that shape is to the eye.

It also has a size preference. A large, spread-out cluster contributes more to J than a small tight one, so the algorithm tends to split genuine large clusters and merge genuine small ones in order to balance the total.

Remark. When the clusters are not blob-shaped, the fix is to change algorithm rather than to tune this one. The k-NN graph methods of §9.3 — spectral clustering, DBSCAN — group points that are connected by chains of short hops and have no convexity assumption at all, which is exactly the case k-means cannot handle. The two families are complementary, and knowing which failure you are looking at is most of the skill.

14.4. Two smaller traps

- **Empty clusters.** If no point is nearest to $\boldsymbol{\mu}_j$, then $\mathcal{C}_j = \varnothing$ and line 8 divides by zero. Implementations detect this and reseed the orphaned centre, usually onto the point currently furthest from its own centre.
- **Feature scaling.** Everything said in §8.2 applies here without modification and for the same reason: J is a sum of squared Euclidean distances, so a feature measured in large units dominates it and silently decides the clustering. Standardize first.

15. In practice

15.1. Where it is used

Application	The vectors, and what a cluster means
Colour quantization	Each pixel of an image is a point in $\mathbb{R}^3$, its red, green and blue values. Cluster them with $k = 256$ and repaint every pixel in its centroid's colour: the image now uses 256 colours instead of millions. This is how a GIF palette is built.
Vector quantization	The same idea in compression generally. The centroids form a *codebook*, and each data vector is transmitted as the index of its nearest centroid rather than as itself.
Customer segmentation	Each customer is a vector of behaviours; the clusters are groups to be treated alike. Probably the most common commercial use.
Document grouping	Documents embedded as vectors; clusters are topics, discovered rather than specified.
Anomaly detection	Cluster the normal data, then score a new point by its distance to the nearest centroid. Large distance, suspicious point — at a cost of $O(kn)$, not $O(Nn)$.
Indexing for search	k-means partitions the space into cells; a nearest-neighbour query then searches only the few cells nearest the query rather than all N points. This is the `IVF` index of §10.2 — k-means is the machinery inside the k-NN speed-up.

The last row is worth pausing on. Part 2 listed IVF among the ways to make nearest-neighbour search fast without saying how the cells were built. They are built by running this algorithm. The two parts of this unit are not merely adjacent topics; one is a component of the other.

15.2. Making it better and faster

k-means++: a better start

Choose the first centre uniformly at random from $\mathcal{X}$. Then choose each subsequent centre at random from $\mathcal{X}$ as well, but with point $\mathbf{x}$ weighted by $D(\mathbf{x})^2$, where $D(\mathbf{x})$ is the distance from $\mathbf{x}$ to the nearest centre already chosen. Points far from everything chosen so far are therefore likely to be picked next, and the initial centres come out spread across the data rather than clumped.

This costs $O(Nkn)$ — one extra iteration's worth — and it buys a genuine guarantee: the expected final J is within a factor of $O(\log k)$ of the true optimum, which is more than plain Lloyd's offers, namely nothing. It has been the default initialisation since Arthur and Vassilvitskii introduced it in 2007.

Skipping distances with the triangle inequality

The assignment step recomputes all Nk distances every iteration, and most of that work is wasted: after the first few passes, almost every point keeps the cluster it had. The following lemma lets those be detected without measuring.

Lemma 15.1 (Elkan)**.** *Let d be a metric and let $\mathbf{x}$, $\mathbf{b}$, $\mathbf{c}$ be points. If $d(\mathbf{b},\mathbf{c}) \geq 2\,d(\mathbf{x},\mathbf{b})$, then $d(\mathbf{x},\mathbf{c}) \geq d(\mathbf{x},\mathbf{b})$.*

Proof. By the triangle inequality **M4** of the vector unit, §14.1, $d(\mathbf{b},\mathbf{c}) \leq d(\mathbf{b},\mathbf{x}) + d(\mathbf{x},\mathbf{c})$. Rearranging and using the hypothesis,

$$d(\mathbf{x},\mathbf{c}) \;\geq\; d(\mathbf{b},\mathbf{c}) - d(\mathbf{x},\mathbf{b}) \;\geq\; 2\,d(\mathbf{x},\mathbf{b}) - d(\mathbf{x},\mathbf{b}) \;=\; d(\mathbf{x},\mathbf{b}). \qquad \square$$

Read $\mathbf{b}$ as the centre $\mathbf{x}$ currently belongs to and $\mathbf{c}$ as some other centre. The lemma says: if the two centres are at least twice as far apart as $\mathbf{x}$ is from its own, then $\mathbf{c}$ cannot be closer, and the distance $d(\mathbf{x},\mathbf{c})$ need never be computed. Since the k centre-to-centre distances change slowly, they can be computed once per iteration — $O(k^2 n)$ — and used to skip a large fraction of the Nk point-to-centre distances. Elkan's algorithm (2003) and Hamerly's (2010) do exactly this, with caches of upper and lower bounds, and are often several times faster while returning *identical* results.

Remark. That lemma is the second time in this unit that a metric axiom has paid for itself computationally — the first was the same pruning applied to k-NN in §10.2. It is worth noticing what kind of fact **M4** turns out to be. In the vector unit it looked like a sanity condition, the formal version of "the direct route is not longer". Here it is the thing that lets a program decline to do arithmetic, and it works for *any* metric, which is why the same trick appears in two different algorithms.

Mini-batch k-means

Instead of using all N points each iteration, draw a random sample of size $b \ll N$ and update the centres from that. Each iteration costs $O(bkn)$ rather than $O(Nkn)$. The result is slightly worse and it arrives orders of magnitude sooner, which is the right trade for datasets that do not fit in memory.

k-medoids

Replace the centroid of line 8 with the medoid of §11.4. The centres are then guaranteed to be actual data points, and the method inherits the medoid's resistance to outliers — at a considerably higher cost, since a medoid needs pairwise distances. The classical algorithm is **PAM** (Partitioning Around Medoids). Use it when averaging your vectors would produce nonsense.

15.3. What the industry uses

Tool	What it is
`scikit-learn`	`KMeans`, with k-means++ initialisation and Elkan's algorithm both available as defaults, plus `MiniBatchKMeans`. The teaching and prototyping standard.
`FAISS`	Meta's `faiss.Kmeans`, built for millions of vectors and GPU execution. Used to build the IVF cells of §10.2.
Spark `MLlib`	Distributed k-means for data spread across a cluster of machines, where the update step becomes a map-reduce over partitions.
`cuML`	NVIDIA's GPU implementation, for when the whole dataset fits in video memory.

Every one of these is line-for-line the pseudocode of §12.3, with a better first line and an assignment step that avoids arithmetic it can prove is unnecessary. The algorithm itself has not changed since Lloyd described it in 1957.

Looking ahead. Both algorithms in this unit reduce to the same two operations: measure distances between vectors, and average vectors. Both were counted with the tools of Part 1 and both live in the floating-point set of §3. What neither can do is change the space the data lives in — and §8.3 already noted that in high dimension the distances they rely on stop discriminating. Fixing that means finding a better basis for the data, which requires the objects of the matrix unit and, eventually, the singular value decomposition. That is the direction the rest of the course takes.